



Direct Speak Integration Guide

Enabling Agentic Workflows with Asynchronous
Text-to-Speech Control



Version 1.0 | December 2025

Technical Integration Documentation

Introduction

This guide describes the **Direct Speak** integration pattern for UneeQ digital humans. This pattern enables advanced agentic workflows where your application maintains full control over the conversation flow and can send multiple sequential speech outputs from a single user input.

Who is this guide for?

- Developers integrating UneeQ digital humans with custom AI/LLM backends
- Teams building agentic applications that require multiple response outputs
- Architects designing conversational AI systems with complex orchestration layers

Key Benefits

- **Full Control:** Your application controls the request-response cycle to the AI backend
- **Multi-Output Support:** Send multiple speech outputs from a single user input (e.g., acknowledgment + answer)
- **Agentic Workflows:** Support for modern AI agents that generate multiple sequential responses
- **Reduced Latency:** Bypass the renderer for orchestration, enabling faster response handling

Traditional Architecture Pattern

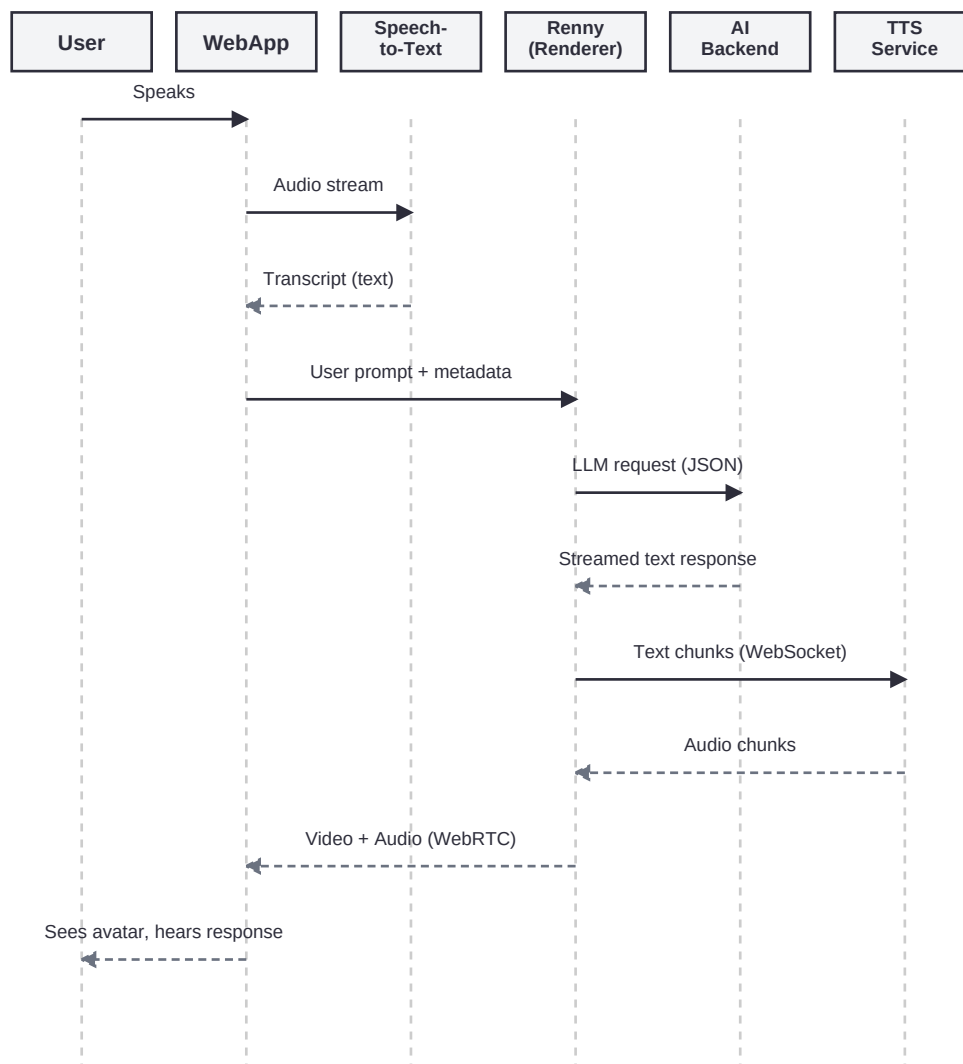
In the traditional integration pattern, the UneeqQ renderer (Renny) acts as the central orchestrator for all conversation flows. User input is transcribed, sent to the renderer, which then forwards requests to the AI backend and handles the response.

Flow Overview

1	User speaks into microphone
2	Speech-to-Text service transcribes audio to text
3	WebApp sends transcript to Renny (UneeQ Renderer)
4	Renny forwards request to AI Backend
5	AI Backend processes (RAG, LLM, APIs) and returns text response
6	Renny receives text and sends to TTS service
7	TTS generates audio, Renny performs lip-sync animation
8	User sees and hears the digital human response

Sequence Diagram

Traditional Architecture Flow



Limitations

- **One Request = One Response:** Each user input produces exactly one output from the AI backend
- **No Mid-Flow Control:** Cannot inject acknowledgments like "OK, let me check..." before the main response
- **Sequential Only:** Cannot handle agentic workflows where multiple outputs are generated asynchronously
- **Tight Coupling:** The renderer is tightly coupled to the AI backend request/response cycle

Direct Speak Architecture Pattern

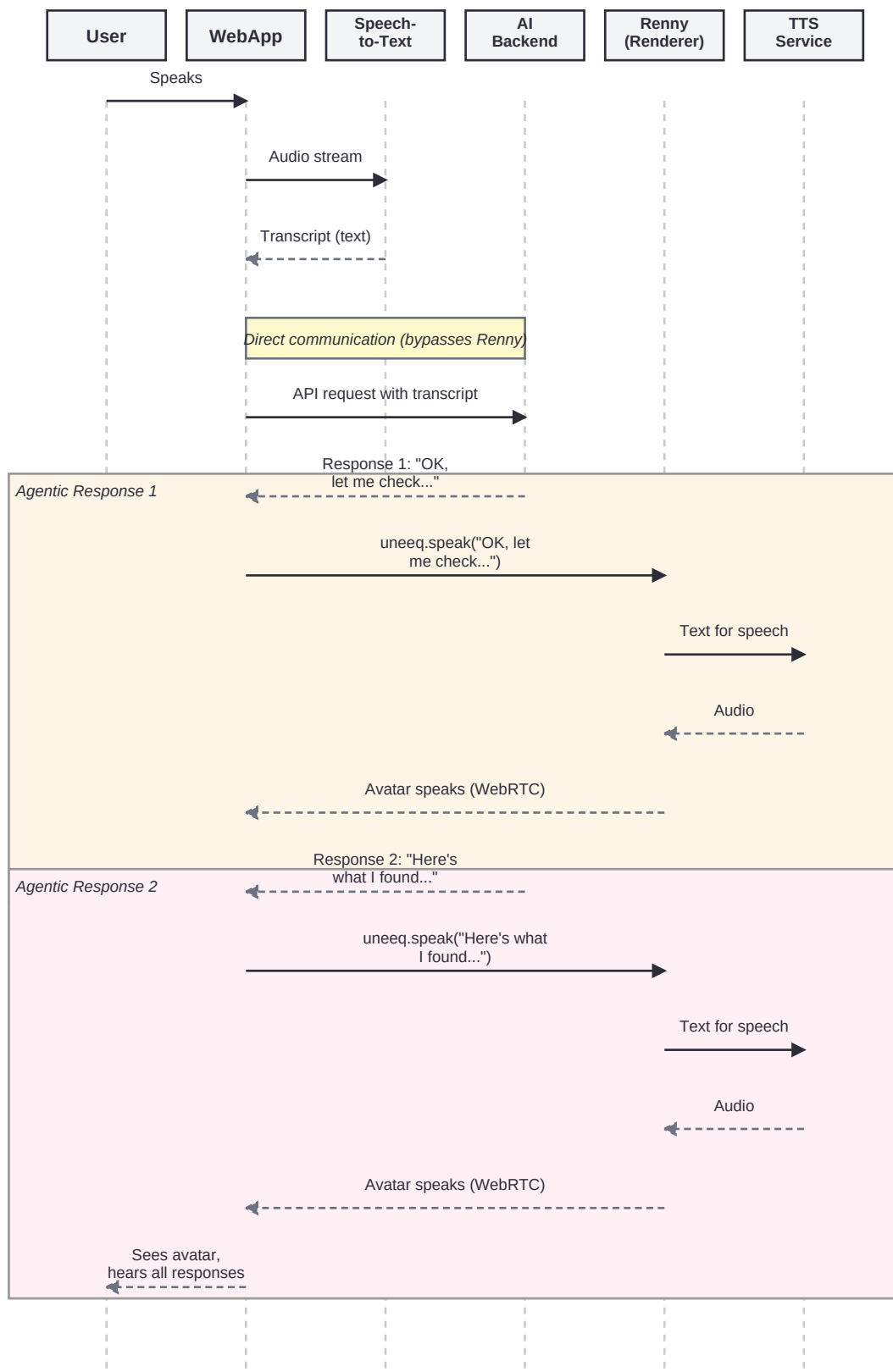
The **Direct Speak** pattern decouples the AI orchestration from the renderer. Your WebApp communicates directly with your AI backend, receives text responses, and then uses the **uneeq.speak()** function to send text directly to the digital human for rendering to speech.

Flow Overview

1	User speaks into microphone
2	Speech-to-Text service transcribes audio to text
3	WebApp sends transcript DIRECTLY to AI Backend (bypasses Renny)
4	AI Backend processes request (RAG, LLM, APIs, agentic workflows)
5	AI Backend returns one or more text responses to WebApp
6	WebApp calls <code>uneeq.speak(response1)</code> → Digital human speaks first part
7	WebApp calls <code>uneeq.speak(response2)</code> → Digital human speaks second part
8	(Repeat for any additional responses from the agent)

Sequence Diagram

Direct Speak Architecture Flow



Key Differences

- **Decoupled Orchestration:** Your WebApp talks directly to your AI backend
- **Multiple Outputs:** Call `uneeq.speak()` multiple times for sequential responses
- **Asynchronous Flow:** Handle streaming responses and send them as they arrive
- **Full Control:** Decide when and what the digital human says

Implementation Guide

The `uneeq.speak()` Function

The `speak()` function is available on the UneeQ SDK instance. It sends a text string directly to the digital human renderer, which then converts it to speech using the configured TTS provider and performs lip-sync animation.

Function Signature

```
// TypeScript
uneeq.speak(speech: string): void

// Parameters:
//   speech - The text string for the digital human to speak
//
// Returns: void (fire-and-forget)
//
// Notes:
//   - Empty strings are ignored
//   - Requires an active session (SessionLive state)
//   - Text is sent via WebRTC data channel to Renny
```

Basic Usage Example

```
// After session is live and you have the uneeq instance
const uneeq = new Uneeq(config);
uneeq.init();

// Listen for session to become live
config.messageHandler = (msg) => {
  if (msg.uneeqMessageType === 'SessionLive') {
    // Session is ready, you can now use speak()
    uneeq.speak("Hello! How can I help you today?");
  }
};
```

Agentic Workflow Example

The following example demonstrates how to integrate with an AI backend that may return multiple responses for a single user query (common in agentic architectures):

```
// Example: Handling multiple responses from an AI agent

async function handleUserInput(transcript: string) {
  // Show that we're processing
  uneeq.speak("OK, let me look into that for you.");

  // Call your AI backend directly (not through Renny)
  const response = await fetch('/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      message: transcript,
    })
  });
}
```



```

        sessionId: currentSessionId
    })
});

const data = await response.json();

// Handle multiple responses from the agent
if (data.responses && Array.isArray(data.responses)) {
    for (const agentResponse of data.responses) {
        // Wait for previous speech to complete (optional)
        await waitForSpeechComplete();

        // Send each response to the digital human
        uneeq.speak(agentResponse.text);
    }
} else {
    // Single response
    uneeq.speak(data.response);
}

// Helper to wait for speech completion
function waitForSpeechComplete(): Promise<void> {
    return new Promise((resolve) => {
        const handler = (msg) => {
            if (msg.uneeqMessageType === 'AvatarStoppedSpeaking') {
                resolve();
            }
        };
        // Add temporary listener
        uneeqMessageHandlers.push(handler);
    });
}

```

Best Practices

Speech Queueing

When sending multiple `speak()` calls, be aware that they are processed sequentially by the renderer. If you send a new `speak()` while the digital human is still speaking, the new text will be queued and spoken after the current speech completes.

Interrupting Speech

```
// To interrupt the digital human mid-speech
uneeq.stopSpeaking();

// Then send new speech
uneeq.speak("Actually, I have an update for you.");
```

Session State Validation

Always ensure the session is in the **SessionLive** state before calling `speak()`. Calls made before the session is live will be ignored.

```
// Track session state
let isSessionLive = false;

config.messageHandler = (msg) => {
  switch (msg.uneeqMessageType) {
    case 'SessionLive':
      isSessionLive = true;
      break;
    case 'SessionEnded':
      isSessionLive = false;
      break;
  }
};

// Safe speak wrapper
function safeSpeak(text: string) {
  if (isSessionLive && text.trim()) {
    uneeq.speak(text);
  } else {
    console.warn('Cannot speak: session not live or empty text');
  }
}
```

Security Considerations

- **API Keys:** Keep AI backend credentials server-side. Never expose API keys in client-side code.
- **Input Sanitization:** Sanitize user input before sending to your AI backend.
- **Rate Limiting:** Implement rate limiting on your API endpoints to prevent abuse.

- **Session Validation:** Validate that requests come from authenticated sessions.

Note: The `speak()` function sends text via WebRTC data channel, which is already secured by the session token. However, your direct API calls to your AI backend should implement their own authentication and authorization.

Architecture Comparison

The following table summarizes the key differences between the traditional architecture and the Direct Speak pattern:

Aspect	Traditional Pattern	Direct Speak Pattern
AI Request Routing	WebApp → Renny → AI Backend	WebApp → AI Backend (direct)
Response Handling	Renny processes response	WebApp controls response flow
Multiple Outputs	Not supported	Fully supported
Agentic Workflows	Limited	Full support
Latency	Additional hop through Renny	Direct, lower latency
Orchestration Control	Renderer-controlled	Application-controlled
TTS Triggering	Automatic by Renny	Explicit via <code>uneeq.speak()</code>
Complexity	Simpler initial setup	Requires API integration

When to Use Each Pattern

Use Traditional Pattern when:

- Simple Q&A; interactions with single responses
- Using UneeQ's built-in conversation platform
- Rapid prototyping without custom backend

Use Direct Speak Pattern when:

- Building agentic AI applications with multiple response outputs
- Need full control over conversation orchestration
- Integrating with custom LLM/AI backends
- Implementing acknowledgments before processing ("OK, let me check...")
- Building complex workflows with conditional responses

Additional Resources

UneeQ SDK Documentation

- **NPM Package:** npmjs.com/package/uneeq-js
- **Developer Portal:** developer.digitalhumans.com
- **Support:** support@uneeq.com

Related SDK Functions

Text-Based Speech Control:

- **`uneeq.speak(text: string)`** - Direct speech control (this guide)
- **`uneeq.chatPrompt(text: string)`** - Send text through traditional flow (via Renny to AI backend)
- **`uneeq.stopSpeaking()`** - Interrupt current speech

Audio Streaming (for pre-generated audio):

- **`uneeq.speakAudio(audio: string | Uint8Array | Blob)`** - Send pre-generated audio for playback
- **`uneeq.openAudioStream()`** - Open an audio streaming session (must call before `speakAudio`)
- **`uneeq.closeAudioStream()`** - Close the audio streaming session
- **`uneeq.interruptAudioStream()`** - Interrupt ongoing audio playback

Digital Human Control:

- **`uneeq.muteDigitalHuman()`** - Mute digital human audio output
- **`uneeq.unmuteDigitalHuman()`** - Unmute digital human audio output
- **`uneeq.setPlaybackSpeed(speed: number)`** - Set TTS playback speed (0.7 - 1.5)



© UneeQ Digital Humans | digitalhumans.com